Programming Principles I

Week 4 · Class 8 · Quiz 2

Quiz 2: 25 minutes

Pen only. No laptop, no phone. Write your name and student ID first.

Programming Principles I

Week 4 · Class 8

Scope, call-by-value and recursion

Where a variable lives · what a function gets · a function that calls itself

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

What Tuesday gave us

1. A function has a **name**, **parameters** and a **return type**
2. `return` hands **one** value back to the caller
3. The call is replaced by that value: `square(5)` becomes `25`
4. `void`: a function that returns nothing
5. Write the function **above** `main`

Recap: what is on the screen?

PREDICT

Two calls in one line.

What does it print?

```
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int total = square(3) + square(4);
    printf("%d\n", total);
    return 0;
}
```

Recap: what is on the screen?

PREDICT

Two calls in one line.

What does it print?

```c
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int total = square(3) + square(4);
    printf("%d\n", total);
    return 0;
}
```

ANSWER

25

`square(3)` is 9, `square(4)` is 16.

Today's roadmap

1. Scope: **where** does a variable live?
2. Call-by-value: **what** does a function get?
3. Recursion: a function that **calls itself**

> Three short parts. Each one answers a question you may have asked on Tuesday.

1 Scope

Where does a variable live?

A variable lives inside its braces

> **A variable made inside a function lives only in that function.**

It is made when the function starts.

It is gone when the function ends.

The part of the program where a name can be used is its **scope**.

main cannot see inside square

```c
int square(int x) {
    int result = x * x;
    return result;
}

int main(void) {
    square(5);
    printf("%d\n", result);
    return 0;
}
```

gcc says

```
error: 'result' undeclared
```

`result` was made inside `square`. `main` has none.

The fix: `int r = square(5);` in `main`. Only the **value** comes back, never the box.

Two boxes with the same name

PREDICT

`main` has an `x`.

`f` has an `x` too.

What are the three lines?

```c
#include <stdio.h>

void f(void) {
    int x = 100;
    printf("in f: x = %d\n", x);
}

int main(void) {
    int x = 5;
    printf("main: x = %d\n", x);
    f();
    printf("main: x = %d\n", x);
    return 0;
}
```

Two boxes with the same name

```c
#include <stdio.h>

void f(void) {
    int x = 100;
    printf("in f: x = %d\n", x);
}

int main(void) {
    int x = 5;
    printf("main: x = %d\n", x);
    f();
    printf("main: x = %d\n", x);
    return 0;
}
```

Output

```
main: x = 5
in f: x = 100
main: x = 5
```

`main`'s `x` and `f`'s `x` are **different boxes**.

Same name. Different scope.

One name, two boxes

main's x

`int x = 5;`

lives in `main`

f's x

`int x = 100;`

lives in `f`

made again at every call

Inside a function you choose names freely. They cannot clash with the names in other functions.

The same rule for loops

```c
int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("%d\n", i);
    }
    printf("After the loop: %d\n", i);
    return 0;
}
```

gcc says

```
error: 'i'
undeclared
```

`i` was made in the `for` brackets.

After the loop, it is gone.

Class 6: `int sum = 0;` inside the loop gave the same error. Same rule.

The scope rule

> **A name lives from the line that makes it to the `}` that closes its block.**

Made in a function: gone when the function ends.

Made in a loop: gone when the loop ends.

Parameters too: `x` in `square(int x)` lives in `square`.

Global variables: you will see them

```
int count = 0;

void tick(void) {
    count++;
}

int main(void) {
    tick();
    tick();
    tick();
    printf("count = %d\n", count);
    return 0;
}
```

Output

```
count = 3
```

Made **outside** every function.

Every function can see it and change it.

Avoid them for now. Use parameters and `return`.

2 Call-by-value

What does a function really get?

A function gets a copy

> **At a call, the value is copied into the parameter.**

The parameter is a new box, inside the function.

Change the copy: the original stays the same.

> Like giving a friend a photocopy of your notes. They can write on it. Your notes stay clean.

Predict: does x change?

PREDICT

`x` is 5.

`double_it` doubles `n`.

What are the two lines?

```c
#include <stdio.h>

void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}

int main(void) {
    int x = 5;
    double_it(x);
    printf("after: x = %d\n", x);
    return 0;
}
```

Run it: x is still 5

```c
#include <stdio.h>

void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}

int main(void) {
    int x = 5;
    double_it(x);
    printf("after: x = %d\n", x);
    return 0;
}
```

Output

```
inside: n = 10
after: x = 5
```

n became 10.

x is still 5. n was a **copy**.

Two boxes, side by side

```c
void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}

int main(void) {
    int x = 5;
    double_it(x);
    printf("after: x = %d\n", x);
```

step	main: x	double_it: n
the call	5	5

The call copies 5 into n. Now there are two boxes, both 5.

Two boxes, side by side

```c
void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}

int main(void) {
    int x = 5;
    double_it(x);
    printf("after: x = %d\n", x);
```

step	main: x	double_it: n
the call	5	5
n = n * 2;	**5**	**10**

n doubles. x is not touched: it is in another box.

Two boxes, side by side

```
void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}

int main(void) {
    int x = 5;
    double_it(x);
    printf("after: x = %d\n", x);
```

step	main: x	double_it: n
the call	5	5
n = n * 2;	5	10
function ends	**5**	**gone**

The function ends and n is gone. x is still 5.

Even with the same name?

PREDICT

The parameter is called x too.

What are the two lines?

```
void add_one(int x) {
    x = x + 1;
    printf("inside: x = %d\n", x);
}

int main(void) {
    int x = 5;
    add_one(x);
    printf("after: x = %d\n", x);
```

Even with the same name?

PREDICT

The parameter is called x too.

What are the two lines?

```
void add_one(int x) {
    x = x + 1;
    printf("inside: x = %d\n", x);
}

int main(void) {
    int x = 5;
    add_one(x);
    printf("after: x = %d\n", x);
```

ANSWER

```
inside: x = 6
after: x = 5
```

The fix: return the new value

```c
#include <stdio.h>

int double_it(int n) {
    return n * 2;
}

int main(void) {
    int x = 5;
    x = double_it(x);
    printf("after: x = %d\n", x);
    return 0;
}
```

Output

```
after: x = 10
```

The function **returns** the doubled value.

`main` keeps it:
`x = double_it(x);`

Predict: swap

PREDICT

`swap` swaps `a` and `b`.

Does it swap `x` and `y`?

```
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("in swap: %d %d\n", a, b);
}

int main(void) {
    int x = 3, y = 7;
    swap(x, y);
    printf("in main: %d %d\n", x, y);
    return 0;
}
```

swap does not swap

```c
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("in swap: %d %d\n", a, b);
}

int main(void) {
    int x = 3, y = 7;
    swap(x, y);
    printf("in main: %d %d\n", x, y);
    return 0;
}
```

Output

```
in swap: 7 3
in main: 3 7
```

The copies were swapped.

x and y were not.

Changing the caller's boxes needs pointers

> **`return` gives back one value. `swap` must change two.**

The real fix uses **pointers**: class 18, after the midterm.

Until then: a function gets copies, and `return` is the way back.

Call-by-value in one slide

The call	Inside	The way back
The **value** is copied into the parameter	Changes change the **copy**	`return`: one value

3

Recursion

A function that calls itself

A function may call itself

> **Solve a big problem by solving a smaller one of the same kind.**

Count down from 3 = say 3, then count down from 2.

It must stop somewhere: at 0, say **Go!**

Like Russian dolls: inside each doll is a smaller one, until the smallest, which does not open.

countdown.c

```c
void countdown(int n) {
    if (n == 0) {
        printf("Go!\n");
        return;
    }
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

Output

```
3
2
1
Go!
```

The base case: when to stop

```
void countdown(int n) {
    if (n == 0) {
        printf("Go!\n");
        return;
    }
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

Base case: at 0, no more calls.

`return;` in a `void` function: stop here.

The step closer

```
void countdown(int n) {
    if (n == 0) {
        printf("Go!\n");
        return;
    }
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

Recursive call: the same job, one smaller.

`n - 1` moves toward the base case.

countdown(3), call by call

call	prints	then calls
`countdown(3)`	3	`countdown(2)`
`countdown(2)`	**2**	**`countdown(1)`**

Each call has its **own** n. The call with 3 is not finished when the call with 2 starts.

countdown(3), call by call

call	prints	then calls
`countdown(3)`	3	`countdown(2)`
`countdown(2)`	2	`countdown(1)`
`countdown(1)`	1	`countdown(0)`
`countdown(0)`	**Go!**	**nothing: base case**

Four calls of one function, each with its own n: 3, 2, 1, 0. The last one stops.

Every recursive function has two parts

Base case	Step closer
A small case you answer **directly**. No more calls.	Call yourself with a **smaller** problem.

No base case, or a step that does not get closer: it never stops.

Factorial two ways

with a loop (class 6)

```
int factorial(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}
```

with recursion

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Both print the same. The right one reads like the maths: n! = n × (n-1)!

factorial: the base case

```c
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main(void) {
    printf("4! = %d\n", factorial(4));
    printf("5! = %d\n", factorial(5));
    return 0;
}
```

Base case: 1! is 1.

n <= 1 catches 0! too: also 1.

factorial: the step closer

```c
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main(void) {
    printf("4! = %d\n", factorial(4));
    printf("5! = %d\n", factorial(5));
    return 0;
}
```

Output

```
4! = 24
5! = 120
```

n! = n × (n-1)!

The * must **wait** for the smaller call.

factorial(4): down and back up

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

call	waits for, or returns
`factorial(4)`	`4 * factorial(3)`

Going **down**: `factorial(4)` needs `factorial(3)` first. It waits.

factorial(4): down and back up

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

call	waits for, or returns
`factorial(4)`	`4 * factorial(3)`
`factorial(3)`	`3 * factorial(2)`
`factorial(2)`	`2 * factorial(1)`
`factorial(1)`	**returns 1**

The base case: `factorial(1)` returns 1 at once. Now the waiting calls can finish.

factorial(4): down and back up

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

call	waits for, or returns
`factorial(4)`	`4 * factorial(3)`
`factorial(3)`	`3 * factorial(2)`
`factorial(2)`	`2 * factorial(1)`
`factorial(1)`	returns 1
`factorial(2)`	2 × 1: returns 2
`factorial(3)`	**3 × 2: returns 6**

Coming back **up**: each waiting call gets its answer and multiplies.

factorial(4): down and back up

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

call	waits for, or returns
`factorial(4)`	`4 * factorial(3)`
`factorial(3)`	`3 * factorial(2)`
`factorial(2)`	`2 * factorial(1)`
`factorial(1)`	returns 1
`factorial(2)`	2 × 1: returns 2
`factorial(3)`	3 × 2: returns 6
`factorial(4)`	**4 × 6: returns 24**

`factorial(4)` returns 24. Four calls down, four returns up.

See it happen: the code

```
int factorial(int n) {
    printf("factorial(%d) starts\n", n);
    if (n <= 1) {
        printf("factorial(%d) returns 1\n", n);
        return 1;
    }
    int result = n * factorial(n - 1);
    printf("factorial(%d) returns %d\n", n, result);
    return result;
}
```

See it happen: the output

```
int main(void) {
    printf("Answer: %d\n", factorial(4));
    return 0;
}
```

Output

```
factorial(4) starts
factorial(3) starts
factorial(2) starts
factorial(1) starts
factorial(1) returns 1
factorial(2) returns 2
factorial(3) returns 6
factorial(4) returns 24
Answer: 24
```

Predict: print after the call

PREDICT

Like countdown, but the `printf` is **after** the call.

What does `count_up(3)` print?

```
void count_up(int n) {
    if (n == 0) {
        return;
    }
    count_up(n - 1);
    printf("%d\n", n);
}

int main(void) {
    count_up(3);
```

Predict: print after the call

PREDICT

Like countdown, but the `printf` is **after** the call.

What does `count_up(3)` print?

```
void count_up(int n) {
    if (n == 0) {
        return;
    }
    count_up(n - 1);
    printf("%d\n", n);
}

int main(void) {
    count_up(3);
```

ANSWER

1, 2, 3, one per line.
Printed on the way back **up**.

No base case: what happens?

CAREFUL

The base case is gone.

What does gcc say?

What happens when it runs?

```
void countdown(int n) {
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

No base case: it crashes

```c
void countdown(int n) {
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

Output

```
3
2
1
0
-1
...
-43130
```

About 43,000 calls, then the memory runs out and the program stops, with **no message**. gcc -Wall warned first: `infinite recursion detected`.

Your turn: sum_to(n)

YOUR TURN

Write `int sum_to(int n)` with recursion.

`sum_to(4)` is 4 + 3 + 2 + 1 = 10.

What is the base case? What is the step closer?

sum_to: one answer

```c
int sum_to(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum_to(n - 1);
}

int main(void) {
    printf("sum_to(4) = %d\n", sum_to(4));
    printf("sum_to(100) = %d\n", sum_to(100));
    return 0;
}
```

Output

```
sum_to(4) = 10
sum_to(100) = 5050
```

Base: the sum of nothing is 0.

Step: `n + sum_to(n - 1)`

One more: power

```c
int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    }
    return base * power(base, exp - 1);
}

int main(void) {
    printf("2 to the 10 = %d\n", power(2, 10));
    printf("3 to the 4 = %d\n", power(3, 4));
    return 0;
}
```

Output

```
2 to the 10 = 1024
3 to the 4 = 81
```

Base: anything to the power 0 is 1.

Why recursion can be slow

```
int fib(int n) {
    calls++;
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

Output

```
fib(30) = 832040
calls: 2692537
```

Two calls inside every call: the same questions, again and again.

A loop needs about 30 steps.

Loop or recursion?

> **Everything today can also be written with a loop.**

Recursion is short when a problem holds a smaller copy of itself.

Today's goal: **read** it and **trace** it on paper.

Today in one slide

1. A variable lives inside its `{ }`: that is its **scope**
2. Same name in two functions: two different boxes
3. A function gets a **copy** of each value
4. To send a new value back: `return` it
5. Recursion: a **base case** and a **step closer**

Before Tuesday

1. **Knowledge check 8** on Blackboard: today's slides
2. **Project 1** (Blackjack): due **Sun 11 October, 23:59**
3. **Quiz 3: Fri 16 October**, classes 7 to 10
4. Today's code: Blackboard, Week 4 module, **Class 8 code**

> Next class: arrays. Many values under one name.